

W2: Querying and Optimizing the Urban Data Platform

Group 4: Gabor, Anggitha, Lalitha

1. Analytical Requirements

1.1 Analytical objectives

To extend the platform by implementing reusable analytical queries, optimizing their performance, and building reusable analytical data products.

1.2 Six analytical requirements, which are:

Analytical Questions	Main Purpose
Monthly taxi demand for each taxi zone	Identify spatial and monthly demand patterns
Average trip distance under weather conditions	Compare trip characteristics for dry, wet and unknown weather
Air quality and taxi demand	Examine the association between PM2.5 and hourly taxi activity
Demand variation under weather conditions	Identify zones where demand changes most between dry and wet conditions
Peak travel hours by weekday	Identify recurring daily and weekly mobility patterns
Monthly taxi demand trends	Measure trip totals, daily averages and month-to month change

2. Analytical Query Design

> The analytical queries are implemented using Spark SQL and operate on the integrated Delta dataset produced by the Week 1 platform. To avoid duplicating common transformation logic across queries, a shared analytical preparation layer is used.

> The trip view provides the common trip-level representation and derives the UTC observation hour and precipitation category. Precipitation is classified as dry when precipitation is zero, wet when precipitation is greater than zero, and unknown when the weather value is missing.

> For calendar-based analyses, the calendar_hours view maps UTC timestamps to local time using the America/New_York timezone. This allows weekday and hourly analyses to reflect local travel behavior while correctly handling daylight-saving-time transitions.

> The hourly_demand view aggregates taxi pickups at hourly level, including hours with zero demand. The zone_weather view provides the zone-level relationship between taxi demand and weather conditions.

> The six analytical queries then operate on these shared views rather than repeating the same temporal and environmental transformations.

Query definitions

1. Monthly taxi demand for each taxi zone $\Rightarrow$ `monthly_zone_demand.sql`
Groups trips by month and pickup zone and counts the number of pickups.
2. Average trip distance under weather conditions $\Rightarrow$ `weather_trip_distance.sql`
Groups valid trip distances by precipitation category and computes the corresponding average.
3. Relationship between air quality and taxi demand $\Rightarrow$ `air_quality_demand.sql`
Calculates the Pearson correlation when sufficient non-constant observations are available.
4. Demand variation under weather conditions $\Rightarrow$ `weather_demand_variation.sql`
Calculates zone-level demand rates for dry and wet conditions and ranks zones by the absolute difference between the two rates.
5. Peak travel hours by weekday $\Rightarrow$ `weekday_peak_hours.sql`
Calculates average hourly demand by local weekday and hour and uses ranking to retain tied peak hours.
6. Monthly taxi demand trends $\Rightarrow$ `monthly_demand_trends.sql`
Aggregates monthly trips and calculates average daily demand and month-over-month change.

3. Short Explanation - Query Optimization Strategy

Four optimization approaches were evaluated in addition to Gold materialization: caching, Adaptive Query Execution (AQE), partition pruning and broadcast joins. Each benchmark was executed five times and median execution time was used for comparison. All optimized cases were checked against their baseline results, and all comparisons reported equivalent results. Detailed execution-time results are reported separately in the benchmark report.

3.1 Caching

Frequently accessed trip data or intermediate results were cached using Spark. This reduces repeated computation and data access for queries that reuse the same data. The trade-off is additional memory/storage usage and cache-population overhead.

3.2 Partition Pruning

The integrated taxi table is partitioned by `pickup\_year` and `pickup\_month`. Queries with time-based filters can therefore read only the relevant partitions, reducing unnecessary data scanning.

3.3 Broadcast Joins

Broadcast joins were investigated for joins between the large taxi-trip dataset and smaller tables such as taxi zones and hourly environmental context. Broadcasting the smaller tables can reduce shuffle and join overhead, at the cost of additional executor memory.

3.4 Adaptive Query Execution (AQE)

AQE was evaluated by comparing execution with AQE enabled and disabled. It allows Spark to adapt execution based on runtime statistics, including shuffle partition handling. Its effectiveness depends on the query and data characteristics.

3.5 Correctness Verification

For each optimization, the optimized query was compared with the baseline result using the query's business keys. Integer and categorical values were compared exactly, while floating-point results were compared using a small numerical tolerance. This ensured that optimization changed the execution strategy without changing the analytical results.

4. Reusable Analytical Data Products

Four analytical products were materialized as Gold Delta tables. The purpose of the Gold layer is to avoid repeatedly scanning and aggregating millions of detailed Silver records when the same analytical summaries are requested frequently.

Data Product	Purpose and Users	Why Materialize
Daily Mobility Summary	Provides daily mobility metrics for mobility analysts and planners.	Supports recurring daily and trend analysis without recomputing the underlying trip-level aggregations.
Taxi Zone Statistics	Provides monthly statistics by pickup zone for spatial and transport analysis.	Allows repeated zone-level analysis without scanning the full integrated dataset.
Weather Impact Summary	Combines zone-level demand with weather conditions for analysts studying weather-related mobility patterns.	Avoids repeatedly joining and aggregating taxi and weather data.
Air Quality Impact Summary	Provides hourly taxi demand together with PM2.5 and temporal/weather context for environmental and mobility analysis.	Makes repeated air-quality analysis more efficient by storing the prepared hourly data.

5. Engineering Decisions and Trade-offs

5.1 SQL/Python separation

Spark SQL is used for the analytical queries and data transformations, while Python is used for orchestration, benchmarking, and generating the analytical data products. This separation keeps the analytical logic reusable and makes the execution workflow easier to manage.

5.2 Silver vs Gold

The analytical queries operate on the integrated Silver dataset, while frequently reused summaries are materialized as Gold data products. Silver provides flexibility for different analyses, whereas Gold reduces repeated computation for commonly used analytical summaries.

5.3 Storage and refresh

The analytical products are stored as Delta tables and regenerated from the integrated dataset. Materialization introduces additional storage and refresh costs, but avoids repeatedly executing expensive joins and aggregations when the same summaries are requested.

5.4 Benchmark design

Query performance is evaluated using repeated executions after a warm-up run. Baseline and optimized queries are compared using the same data and query results, while `EXPLAIN FORMATTED` is used to inspect changes in the physical execution plan. Correctness is checked separately from execution time.

5.5 Scaling to ten cities

For expansion to ten cities, the analytical model should include a `city_id` and use the appropriate local timezone for calendar-based analysis. Partitioning and broadcast-table sizes should be reassessed as data volume increases, and incremental refresh could be considered for materialized products.

6. Conclusion

The Week 2 design extends the Week 1 platform with reusable Spark SQL analyses and materialized Delta data products. Shared analytical views provide consistent temporal and environmental processing, while caching, partition pruning, broadcast joins, and AQE provide alternative strategies for improving query execution. The separation between on-demand queries and reusable data products also provides a foundation for scaling the platform to additional cities.